

IDS Project - Next Steps & Roadmap

Table of Contents

- [Immediate Priorities](#)
 - [Short-term Goals \(1-2 months\)](#)
 - [Medium-term Goals \(3-6 months\)](#)
 - [Long-term Vision \(6+ months\)](#)
 - [Research Opportunities](#)
 - [Technical Debt](#)
-

Immediate Priorities

PHASE 1: Model Testing & Validation (Week 1-2)

1.1 Test All ML Models

Goal: Validate all existing models and log comprehensive metrics

- ☐ **Test Individual Models**
 - ☐ Random Forest (CICIDS2017) - [ML Models/random_forest_model_2017.joblib](#)
 - Load model and verify it works with current pipeline
 - Test with CICIDS2017 dataset samples
 - Log: Accuracy, Precision, Recall, F1-score, Confidence scores
 - ☐ LightGBM (CICIDS2018) - [ML Models/lgb_model_2018.joblib](#)
 - Load model and verify it works with current pipeline
 - Test with CICIDS2018 dataset samples
 - Log: Accuracy, Precision, Recall, F1-score, Confidence scores
- ☐ **Test Ensemble Models**
 - ☐ Adaptive Ensemble - [utils/adaptive_ensemble_predictor.py](#)
 - Test voting mechanism (RF + LightGBM)
 - Compare ensemble vs individual model performance
 - Log: Combined accuracy, confidence distributions, decision weights
 - ☐ Standard Ensemble - [tests/test_ensemble_model.py](#)
 - Verify ensemble integration with pipeline
 - Test weighted averaging
 - Log: Performance improvements over single models
- ☐ **Pipeline Integration Testing**

- ☐ Verify each model receives correct 65 CICIDS features
- ☐ Test feature mapping (65 → 34) for each model
- ☐ Confirm Kafka integration for all models
- ☐ Test model switching/hot-swapping capability

1.2 📊 Metrics Collection & Logging

Create comprehensive logging system for model performance

- ☐ **Model Performance Metrics**
 - ☐ Accuracy (per attack type)
 - ☐ Precision & Recall (per class)
 - ☐ F1-score (weighted & macro)
 - ☐ Confusion matrices
 - ☐ ROC-AUC scores
 - ☐ Confidence score distributions (0.0-1.0)
 - ☐ False positive/negative rates
- ☐ **Attack Detection Breakdown**
 - ☐ BENIGN detection rate
 - ☐ DDoS detection (SYN, UDP, HTTP flood)
 - ☐ Port Scan detection (Nmap patterns)
 - ☐ Brute Force detection (SSH, FTP)
 - ☐ Web Attack detection (SQL injection, XSS)
 - ☐ Botnet detection (C&C patterns)
 - ☐ Infiltration detection
- ☐ **Create Metrics Logger Script**

```
# New script: utils/model_metrics_logger.py
- Real-time metric collection
- CSV/JSON output for analysis
- Per-model comparison dashboard
- Confidence threshold analysis
```

- ☐ **Generate Model Comparison Report**
 - Side-by-side performance comparison
 - Best model per attack type
 - Ensemble improvement quantification
 - Recommendations for production use

PHASE 2: Pipeline Performance Testing (Week 2-3)

2.1 ✂ AF_PACKET Mode Benchmarking

- ☐ **Throughput Testing**
 - ☐ Packets Per Second (PPS) - Measure max processing rate
 - ☐ Mbps throughput - Network bandwidth handling
 - ☐ Concurrent connections - Maximum simultaneous flows
 - ☐ Queue sizes - Kafka, Suricata buffer analysis
- ☐ **Latency Measurements**
 - ☐ Packet capture → Suricata detection (µs)
 - ☐ Suricata → Kafka publish (ms)
 - ☐ Kafka → ML consumer read (ms)
 - ☐ ML feature extraction → prediction (ms)
 - ☐ Total end-to-end latency (ms)
 - ☐ 95th/99th percentile latency
- ☐ **Accuracy & Quality**
 - ☐ Packet drop rate (%)
 - ☐ False positive rate (per 1000 flows)
 - ☐ False negative rate (missed attacks)
 - ☐ Detection accuracy per traffic volume
- ☐ **Resource Usage**
 - ☐ CPU utilization (per core, average %)
 - ☐ Memory consumption (RSS, heap)
 - ☐ Disk I/O (read/write MB/s)
 - ☐ Network I/O (bytes in/out)
 - ☐ System load average
- ☐ **Create Benchmark Script**

```
# New: tests/benchmark_afpacket.py
- Automated testing with various traffic loads
- Generate performance report
- Identify bottlenecks
```

2.2 🚀 DPDK Mode Benchmarking (If hardware available)

- ☐ **High-Performance Metrics**
 - ☐ PPS (target: 10M+ pps)
 - ☐ Multi-Gbps throughput (1/10/40 Gbps)
 - ☐ Packet loss at high loads
 - ☐ CPU core utilization per DPDK worker
 - ☐ Hugepage usage efficiency

- ☐ **DPDK-Specific Testing**
 - ☐ Interface binding/unbinding stability
 - ☐ PMD (Poll Mode Driver) performance
 - ☐ Zero-copy efficiency
 - ☐ Multi-queue performance
- ☐ **Comparison Report**
 - ☐ AF_PACKET vs DPDK performance matrix
 - ☐ Cost/benefit analysis
 - ☐ Use case recommendations

2.3 Performance Visualization

- ☐ **Create Real-time Monitoring**
 - ☐ Live metrics dashboard (terminal UI)
 - ☐ Performance graphs (matplotlib/plotly)
 - ☐ Bottleneck identification
 - ☐ Alerting for performance degradation

PHASE 3: Dashboard Architecture Design (Week 3-4)

3.1 Dashboard Requirements & Design

- ☐ **Define Dashboard Architecture**
 - ☐ Choose technology stack:
 - **Option A:** Elasticsearch + Kibana (ELK Stack)
 - **Option B:** Prometheus + Grafana
 - **Option C:** InfluxDB + Grafana
 - **Option D:** Custom (React + WebSocket + D3.js)
 - ☐ Data flow architecture:

Kafka (ml-predictions) → Data Sink → Database → Visualization

- ☐ **Design Dashboard Components**

Real-time Monitoring Panel:

- ☐ Live threat map (GeoIP-based)
- ☐ Attack type distribution (pie/bar chart)
- ☐ Timeline of alerts (last 1h/24h/7d)
- ☐ Top 10 attackers (IP addresses)
- ☐ Top 10 targets (internal IPs)

- ☐ Threat level gauge (LOW/MEDIUM/HIGH/CRITICAL)

ML Model Performance Panel:

- ☐ Model accuracy metrics (real-time)
- ☐ Confidence score distribution
- ☐ Prediction rate (predictions/sec)
- ☐ Model comparison (RF vs LightGBM vs Ensemble)
- ☐ Feature importance visualization

Pipeline Health Panel:

- ☐ Component status (Kafka, Suricata, ML Consumer)
- ☐ Throughput graphs (PPS, Mbps)
- ☐ Latency graphs (P50, P95, P99)
- ☐ Resource usage (CPU, Memory, Network)
- ☐ Error rates and exceptions

Attack Deep-Dive Panel:

- ☐ Per-attack-type statistics
- ☐ Attack timeline visualization
- ☐ Packet details (on-demand PCAP viewer)
- ☐ Correlation graphs (multi-stage attacks)

3.2 ✂ Technology Stack Selection

- ☐ **Evaluate Options:**

ELK Stack (Elasticsearch + Kibana):

- ☒ Rich visualization options
- ☒ Great for log analysis
- ☒ Built-in alerting
- × Resource-intensive
- × Complex setup

Prometheus + Grafana:

- ☒ Excellent for metrics
- ☒ Lightweight
- ☒ Easy setup
- × Not ideal for logs
- × Time-series focused

Custom Dashboard:

- ☒ Full control
- ☒ Tailored to needs
- × Development time

- × Maintenance burden
- ☐ **Make Technology Decision**
 - ☐ Document pros/cons
 - ☐ Consider team skills
 - ☐ Evaluate resource constraints
 - ☐ Choose stack and document rationale

3.3 🗺️ Architecture Documentation

- ☐ **Create Dashboard Architecture Document**
 - Data ingestion pipeline
 - Storage architecture
 - Query optimization strategies
 - Visualization component layout
 - API design (if custom dashboard)
 - Security considerations (auth, access control)
- ☐ **Create Implementation Plan**
 - ☐ Phase 1: Data ingestion setup
 - ☐ Phase 2: Database/storage setup
 - ☐ Phase 3: Basic dashboards
 - ☐ Phase 4: Advanced visualizations
 - ☐ Phase 5: Alerting integration

PHASE 4: Documentation & Cleanup (Week 4)

- ☐ **Update Documentation**
 - ☐ Model performance comparison report
 - ☐ Pipeline benchmark results
 - ☐ Dashboard architecture guide
 - ☐ Updated README with findings
- ☐ **Code Cleanup**
 - ☐ Remove unused test scripts
 - ☐ Consolidate duplicate code
 - ☐ Add docstrings to new scripts
 - ☐ Update requirements.txt

Summary: Immediate Action Plan (4 Weeks)

Week 1: ML Model Testing

- Test RF, LightGBM, Ensemble models
- Collect comprehensive metrics
- Generate model comparison report

Week 2: AF_PACKET Performance

- Benchmark PPS, latency, accuracy
- Measure resource usage
- Document performance baseline

Week 3: DPDK Testing + Dashboard Design

- DPDK benchmarks (if available)
- Design dashboard architecture
- Choose technology stack

Week 4: Dashboard Implementation Planning

- Create detailed implementation plan
- Document architecture decisions
- Update all documentation

Short-term Goals (1-2 months)

4. Machine Learning Enhancements

4.1 Ensemble Model Optimization

- ☐ **Implement Robust Ensemble**
 - ☐ Voting ensemble (RF + LightGBM + XGBoost)
 - ☐ Weighted average based on historical confidence scores
 - ☐ Optimize `utils/adaptive_ensemble_predictor.py`
 - ☐ Test different voting strategies (hard, soft, weighted)
 - ☐ Measure ensemble improvement over single models
- ☐ **Confidence Calibration**
 - ☐ Analyze confidence score distributions
 - ☐ Implement confidence calibration (Platt scaling, isotonic regression)
 - ☐ Set optimal confidence thresholds per attack type
 - ☐ Reduce false positives with calibrated scores
- ☐ **Ensemble Performance Testing**
 - ☐ Compare ensemble vs individual models on test set
 - ☐ Measure improvement in accuracy, precision, recall
 - ☐ Test on edge cases and rare attacks
 - ☐ Document when ensemble performs better/worse

4.2 Online Learning Implementation

- ☐ **Incremental Learning System**
 - ☐ Implement online learning for Random Forest
 - ☐ Implement online learning for LightGBM
 - ☐ Create feedback loop for model updates
 - ☐ Handle concept drift (evolving attack patterns)
- ☐ **Active Learning Pipeline**
 - ☐ Identify uncertain predictions (low confidence)
 - ☐ Queue samples for manual labeling
 - ☐ Retrain models with new labeled data
 - ☐ Track model performance over time
- ☐ **Model Update Strategy**
 - ☐ Define update frequency (daily, weekly)
 - ☐ Implement A/B testing for new models
 - ☐ Version control for models (MLflow)
 - ☐ Rollback strategy if performance degrades

4.3 Feature Engineering for CICIDS Datasets

Goal: Optimize features specifically for CICIDS2017/2018 datasets

- ☐ **Feature Analysis**
 - ☐ Analyze feature importance for RF and LightGBM models
 - ☐ Identify redundant/correlated features
 - ☐ Test feature selection algorithms (RFE, mutual information)
 - ☐ Document most important features per attack type
- ☐ **Feature Optimization**
 - ☐ Remove low-importance features (< 0.01 importance)
 - ☐ Create new derived features from existing ones
 - ☐ Test polynomial features for non-linear relationships
 - ☐ Optimize feature extraction performance
- ☐ **CICIDS-Specific Features**
 - ☐ Verify all 65 CICIDS2017 features are extracted correctly
 - ☐ Add missing features if any
 - ☐ Handle missing/null values properly
 - ☐ Normalize/standardize features correctly
- ☐ **Cross-Dataset Testing**
 - ☐ Test CICIDS2017 model on CICIDS2018 data

- ☐ Test CICIDS2018 model on CICIDS2017 data
- ☐ Measure generalization performance
- ☐ Identify dataset-specific biases

Note: Focus on improving current models with CICIDS datasets. Advanced techniques (deep learning, graph features, etc.) moved to medium-term goals.

5. Detection Capabilities

Advanced Attack Detection

- ☐ **Zero-Day Detection**
 - Anomaly detection using autoencoders
 - One-class SVM for outlier detection
 - Isolation forests for novel attacks
- ☐ **Advanced Persistent Threats (APT)**
 - Long-term behavior profiling
 - Multi-stage attack correlation
 - Lateral movement detection
- ☐ **Encrypted Traffic Analysis**
 - TLS fingerprinting (JA3/JA3S)
 - Encrypted malware detection (timing, size patterns)
 - DNS-over-HTTPS (DoH) analysis
- ☐ **IoT Attack Detection**
 - Mirai botnet patterns
 - Device fingerprinting
 - Anomalous IoT behavior

PROF

Attack Response

- ☐ **Automated Response System**
 - Firewall rule generation (iptables/nftables)
 - Automatic IP blocking
 - VLAN isolation for compromised hosts
- ☐ **Threat Intelligence Integration**
 - AlienVault OTX feeds
 - Abuse.ch feeds
 - Custom blacklist/whitelist management
- ☐ **SIEM Integration**

- Splunk connector
- IBM QRadar integration
- ArcSight compatibility

6. 🏗️ Architecture Improvements

Scalability

- ☐ **Distributed Processing**
 - Multi-node Kafka cluster
 - Kafka Streams for stateful processing
 - Horizontal scaling of ML consumers
- ☐ **Load Balancing**
 - Multiple Suricata instances
 - Traffic mirroring/SPAN port configuration
 - Round-robin packet distribution
- ☐ **Database Backend**
 - PostgreSQL for structured alerts
 - TimescaleDB for time-series data
 - Redis for caching and fast lookups

Reliability

- ☐ **High Availability**
 - Kafka replication (3+ brokers)
 - Suricata failover configuration
 - ML consumer redundancy
- ☐ **Data Persistence**
 - Long-term alert storage (S3/MinIO)
 - Backup and recovery procedures
 - PCAP archiving for forensics
- ☐ **Error Handling**
 - Circuit breakers for external services
 - Dead letter queues for failed messages
 - Graceful degradation

PROF

Medium-term Goals (3-6 months)

7. 🌐 Advanced Features

Network Forensics

- ☐ **Full Packet Capture**
 - Triggered PCAP capture for high-threat events
 - PCAP-over-IP streaming
 - PCAP analysis tools (Wireshark automation)
- ☐ **Session Reconstruction**
 - TCP stream reassembly
 - HTTP transaction extraction
 - File carving from network traffic
- ☐ **Behavioral Analysis**
 - User and Entity Behavior Analytics (UEBA)
 - Baseline normal behavior per host
 - Anomaly scoring per entity

Threat Hunting

- ☐ **Query Interface**
 - SQL-like query language for alerts
 - Interactive threat hunting dashboard
 - Saved queries and reports
- ☐ **Correlation Engine**
 - Multi-event correlation rules
 - Attack chain detection
 - Kill chain mapping (Lockheed Martin framework)
- ☐ **Threat Indicators**
 - IOC (Indicators of Compromise) database
 - STIX/TAXII integration
 - Custom indicator management

—
PROF

8. Deep Learning Models

Neural Network Architectures

- ☐ **Convolutional Neural Networks (CNN)**
 - Treat packets as images (pixel-based representation)
 - 1D-CNN for sequential packet features
 - Learn hierarchical features automatically
- ☐ **Recurrent Neural Networks (RNN/LSTM)**

- Model temporal dependencies
- Sequence-to-sequence learning
- Predict next-event in attack sequence
- ☐ **Graph Neural Networks (GNN)**
 - Learn from network topology
 - Node classification (host threat level)
 - Link prediction (lateral movement)
- ☐ **Transformer Models**
 - Attention mechanism for traffic analysis
 - BERT-style pre-training on network flows
 - Few-shot learning for rare attacks

Advanced ML Techniques

- ☐ **Federated Learning**
 - Train models across multiple organizations
 - Privacy-preserving collaborative learning
 - Share threat intelligence without sharing data
- ☐ **Adversarial Machine Learning**
 - Test model robustness against adversarial attacks
 - Evasion attack detection
 - Generate adversarial examples for training
- ☐ **Explainable AI (XAI)**
 - SHAP values for feature importance
 - LIME for local interpretability
 - Attention visualization for transformers

PROF

9. 🌐 Deployment Options

Cloud Deployment

- ☐ **AWS Architecture**
 - EC2 for compute
 - MSK (Managed Kafka)
 - S3 for storage
 - Lambda for serverless processing
- ☐ **Azure Architecture**
 - Virtual Machines
 - Event Hubs (Kafka-compatible)

- Blob Storage
- Azure ML for model serving
- ☐ **GCP Architecture**
 - Compute Engine
 - Pub/Sub (Kafka alternative)
 - Cloud Storage
 - Vertex AI for ML

Containerization

- ☐ **Docker Compose**
 - Multi-container orchestration
 - Development environment setup
 - Easy deployment
- ☐ **Kubernetes Deployment**
 - Production-grade orchestration
 - Auto-scaling based on traffic
 - Helm charts for package management
- ☐ **Edge Deployment**
 - Lightweight containers for IoT gateways
 - Edge ML inference (TensorFlow Lite)
 - Fog computing architecture

Long-term Vision (6+ months)

10. 🚀 Enterprise Features

PROF

Multi-Tenancy

- ☐ **Organization Management**
 - Separate namespaces per customer
 - Isolated data streams
 - Per-tenant model customization
- ☐ **Role-Based Access Control (RBAC)**
 - Admin, analyst, viewer roles
 - Fine-grained permissions
 - Audit logging

Compliance & Reporting

- ☐ **Compliance Frameworks**
 - GDPR compliance (data retention, privacy)
 - PCI-DSS reporting
 - HIPAA audit logs
 - ISO 27001 documentation
- ☐ **Automated Reports**
 - Executive dashboards
 - Weekly threat summaries
 - Incident response reports
 - Compliance attestations

Commercial Features

- ☐ **Licensing System**
 - Subscription management
 - Usage tracking
 - Feature gating
- ☐ **Support Infrastructure**
 - Ticketing system integration
 - Remote diagnostics
 - Update management

11. 🧠 AI-Driven Security Operations

Autonomous Security

- ☐ **Self-Healing Systems**
 - Automatic remediation of detected threats
 - Policy learning from analyst actions
 - Continuous optimization
- ☐ **Predictive Security**
 - Forecast attack likelihood
 - Vulnerability prioritization
 - Risk scoring predictions
- ☐ **Natural Language Interface**
 - ChatGPT-style threat hunting queries
 - Voice-activated security operations
 - Automated incident reporting

Security Orchestration

- ☐ **SOAR Integration** (Security Orchestration, Automation, Response)
 - Phantom/Splunk SOAR
 - Cortex XSOAR
 - TheHive integration
 - ☐ **Playbook Automation**
 - Automated incident response workflows
 - Runbook execution
 - Case management
-

Research Opportunities

12. 📖 Academic Research

Publications

- ☐ **Conference Papers**
 - IEEE S&P, USENIX Security, NDSS
 - ACM CCS, ACSAC
 - Research on novel ML techniques for IDS
- ☐ **Journal Articles**
 - IEEE Transactions on Information Forensics and Security
 - Computers & Security
 - Journal of Cybersecurity

Research Topics

- ☐ **Transfer Learning for IDS**
 - Pre-train on large public datasets
 - Fine-tune on organization-specific traffic
 - Domain adaptation techniques
- ☐ **Adversarial Robustness**
 - Evasion attacks on ML-based IDS
 - Defense mechanisms
 - Certified robustness bounds
- ☐ **Privacy-Preserving IDS**
 - Homomorphic encryption for traffic analysis
 - Differential privacy guarantees

- Secure multi-party computation
- ☐ **Quantum-Resistant IDS**
 - Post-quantum cryptography integration
 - Quantum machine learning models
 - Quantum-safe protocols

13. 🤝 Open Source Community

Community Building

- ☐ **GitHub Repository Management**
 - Issue templates
 - Contributing guidelines
 - Code of conduct
- ☐ **Documentation**
 - Developer guide
 - API reference
 - Architecture documentation
- ☐ **Community Engagement**
 - Discord/Slack community
 - Monthly community calls
 - Bounty program for contributions

Ecosystem Growth

- ☐ **Plugin System**
 - Custom detection plugins
 - Third-party integrations
 - Protocol parsers
- ☐ **Marketplace**
 - Pre-trained models
 - Detection rules
 - Dashboard templates

—
PROF

Technical Debt

14. 🛠️ Code Quality

Refactoring

- ☐ **Type Hints**
 - Add Python type annotations
 - Use mypy for static type checking
- ☐ **Code Documentation**
 - Docstrings for all functions
 - Inline comments for complex logic
 - Architecture Decision Records (ADRs)
- ☐ **Code Style**
 - Black formatter
 - Pylint/Flake8 linting
 - Pre-commit hooks

Testing

- ☐ **Unit Tests**
 - 80%+ code coverage
 - Mock external dependencies
 - Fast test suite (< 1 minute)
- ☐ **Integration Tests**
 - End-to-end pipeline tests
 - Kafka integration tests
 - Database tests
- ☐ **Performance Tests**
 - Load testing (locust/JMeter)
 - Stress testing
 - Regression benchmarks

—
PROF

CI/CD

- ☐ **GitHub Actions**
 - Automated testing on push
 - Linting and formatting checks
 - Security scanning (Snyk, Dependabot)
- ☐ **Deployment Pipeline**
 - Automated builds
 - Staging environment
 - Canary deployments

15. Security

Application Security

- ☐ **Dependency Scanning**
 - Regular vulnerability scans
 - Automated dependency updates
 - SBOM (Software Bill of Materials)
- ☐ **Secret Management**
 - Vault/AWS Secrets Manager
 - Environment variable encryption
 - Key rotation policies
- ☐ **Secure Configuration**
 - TLS/SSL for all communications
 - Authentication for Kafka
 - Network segmentation

Operational Security

- ☐ **Logging & Auditing**
 - Centralized logging (ELK stack)
 - Security event logging
 - Tamper-proof audit trails
- ☐ **Incident Response**
 - Incident response playbook
 - Disaster recovery plan
 - Backup and restore procedures

PROF

Priority Matrix

High Priority (Do First)

1. ☒ Testing & validation of cleaned codebase
2. ☒ Real-time dashboard setup
3. ☒ Performance benchmarking

Medium Priority (Do Next)

5. Advanced attack detection capabilities
6. Distributed architecture implementation
7. Deep learning model exploration

8. Cloud deployment options

Low Priority (Nice to Have)

- 9. Enterprise multi-tenancy features
 - 10. Academic research publications
 - 11. Commercial licensing system
 - 12. Quantum-resistant features
-

Timeline Estimate

Month 1-2: Foundation

-  Code cleanup (DONE)
-  Testing and validation
-  Basic dashboard
-  Documentation updates

Month 3-4: Enhancement

-  Model improvements
-  Advanced detection
-  Scalability improvements
-  Performance optimization

Month 5-6: Expansion

-  Deep learning models
-  Cloud deployment
-  SIEM integration
-  Threat intelligence feeds

—
PROF

Month 7-12: Production

-  Enterprise features
 -  High availability setup
 -  Compliance frameworks
 -  Commercial readiness
-

Success Metrics

Technical Metrics

-  **Detection Accuracy:** > 99% (currently 99.2-99.5%)
-  **False Positive Rate:** < 1%
-  **Throughput:** 10 Gbps (DPDK mode)
-  **Latency:** < 100ms end-to-end

-  **Availability:** 99.9% uptime

Business Metrics

-  **Deployment:** 10+ production deployments
 -  **Community:** 1000+ GitHub stars
 -  **Contributors:** 50+ active contributors
 -  **Publications:** 3+ research papers
-

Getting Started with Next Steps

For Contributors

1. **Pick a task** from the "Immediate Priorities" section
2. **Create an issue** on GitHub with your proposal
3. **Fork the repository** and create a feature branch
4. **Implement the feature** with tests and documentation
5. **Submit a pull request** for review

For Researchers

1. **Review the research opportunities** section
2. **Contact the project maintainers** to discuss collaboration
3. **Access the datasets** and pre-trained models
4. **Contribute findings** back to the project

For Users

1. **Test the current system** and provide feedback
2. **Report bugs** and feature requests on GitHub
3. **Share use cases** and deployment experiences
4. **Contribute to documentation** improvements

PROF

Resources

Learning Materials

-  [Suricata Documentation](#)
-  [DPDK Programming Guide](#)
-  [Kafka Documentation](#)
-  [CICIDS2017 Dataset Paper](#)

Tools & Frameworks

-  [MLflow](#) - ML experiment tracking
-  [Grafana](#) - Monitoring dashboards
-  [Elasticsearch](#) - Log analysis

-  TensorFlow - Deep learning

Communities

-  Suricata Community
-  DPDK Community
-  ML for Cybersecurity

Conclusion

This IDS project has tremendous potential for growth and impact. The cleaned codebase provides a solid foundation for implementing these next steps.

Priority focus areas:

1.  Validate and test the current implementation
2.  Add visualization and monitoring
3.  Enhance ML models with latest techniques
4.  Scale to production-grade deployment

The combination of traditional signature-based detection (Suricata) with machine learning creates a powerful hybrid approach that can detect both known and unknown threats.

Let's build the future of network security! 

Questions or Ideas?

-  Open an issue on GitHub
-  Join the discussion forum
-  Submit a feature request
-  Contribute to the project

Happy coding and stay secure! 